SC4002 Natural Language Processing
AY26-27 Sem1 Group Assignment

Instruction

- This is a group assignment. Each group consists of 5 to 6 students.
- The assignment constitutes 30% of your total grade for this course.
- Please submit your assignment solution via NTULearn → Group Assignment Submission by Sunday, 15th November at 11:59pm SGT. For late submission, there will be a 5% point deduction each calendar day after the deadline. You are advised to submit not more than three times to the system, and only the last submission will be graded and time-stamped.
- Please name your file as “Group_<ID>.zip”
- Please list all the full names of the group members and your group ID in the cover page of the report. All members in the same group will receive the same grade. However, contributions of each individual member to the assignment should be clearly indicated in the cover page of the report.

Summary of Project Instructions

In groups of 5–6, build a system that identifies a user’s intent and extracts the slots in an English assistant request. It involves two stages:

- **Stage 1**: develop a baseline independently, without generative AI assistance.
- **Stage 2**: collaborate with AI to diagnose weaknesses, implement improvements

Evaluate the resulting models from the above two stages. Submit reproducible notebook evidence, chat history with AI, and a report of no more than two pages.

1. Task and required outputs

- **Task 1 - Intent classification**: predict one of the 60 MASSIVE intent labels for each utterance. An example intent is “alarm_set”.
- **Task 2 - Slot filling**: extract slot spans and classify their corresponding slot types. There are 55 slot types. An example [slot type: slot span] is [time: five am].

Both tasks are required in the system built at every stage. Every system must produce an intent label and the set of [slot type: slot span] pairs for each input utterance.

2. Dataset and official splits

Use MASSIVE 1.0, English data (en-US), from the original release. Download the archive below, extract it, and use **1.0/data/en-US.jsonl**. Partition records into train, dev, and test, using the existing partition field; dev is the validation split. These released labelled splits contain 16,521 examples in total.

Split	Partition	Examples	Permitted use
Training	train	11,514	Fit model parameters and vocabulary
Validation	dev	2,033	Tune settings and select final model for evaluation
Test	test	2,974	Final evaluation after the model is frozen

Official download:
https://amazon-massive-nlu-dataset.s3.amazonaws.com/amazon-massive-dataset-1.0.tar.gz

Official documentation: https://github.com/alexa/massive

Keep every example in its original split. Do not re-split, add, or remove examples. Use training split to train the model, validation split to tune hyper-parameters, and test split to perform final evaluation. Do not augment the training data with other datasets.

For each example, the only input to the model is “utt”. “intent” gives the label for the first task: intent classification. “annot_utt” contains the label for the second task: slot filling.

Example: *utt = “wake me up at five am this week”; intent = alarm_set; annot_utt = “wake me up at [time : five am] [date : this week]”.*

3. Permitted tools and model scope

- There are no prescribed model architectures in either stage. **No pretrained models are allowed (except word vectors).** You may use any deep learning models capped at **20M**. The improved system in the second stage may refine the system you have built in the first stage or use a different architecture.
- Do not use models already trained or fine-tuned on MASSIVE or related tasks.
- Train or fine-tune each system yourselves using only the official MASSIVE English training split. No external or AI-generated task examples are allowed.

4. Evaluation metrics

Report all metrics as percentages with two decimal places.

Metric	Definition
Intent accuracy	100 × correct intent predictions / number of utterances.
Slot micro F1	F1 score on exact [slot type: slot span] match, pooled across all examples. A true positive requires matching both slot type and slot span. F1 = 100 × 2TP / (2TP + FP + FN).

5. Submission package & Required steps

Required item	Contents
main.ipynb	All the source codes for two stages, including training logs, output, and evaluation results.
ai_collaboration.md/txt	Chat history with AI demonstrating your collaboration
report.pdf	Maximum two-page report (excluding the cover page)

What to include in main.ipynb

Note: Do not type scores manually, fabricate logs, or clear outputs before submission. If being spotted, your group will receive zero marks.

- **Load and verify the dataset**: Load MASSIVE English using the original splits. Print and assert 11,514 training, 2,033 validation, and 2,974 test examples.
- **Implement preprocessing**: Convert raw dataset into suitable inputs and labels for your model
- **Implement the shared evaluation functions**: Code functions which compute intent accuracy and slot micro F1. Use identical scoring rules for every experiment.
- **Define the independent baseline (Stage-1 model)**: Explain the architecture, inputs, outputs, loss, optimizer. Print total parameter count in the pipeline and verify the total count is within **20M**. Complete this stage without generative AI assistance.
- **Train and select the baseline checkpoint**: Train using training data only. Retain epoch-level training/validation losses, validation results for each epoch, elapsed time, and learning curves. Save the checkpoint with the highest validation score (intent accuracy, slot micro F1, or combined, depending on your choice). Print its corresponding epoch index, validation score, and checkpoint filename.
- **Perform final evaluation on baseline checkpoint**: Load the checkpoint saved from the previous step. Without further training, evaluate the checkpoint on the validation and test splits. Your validation score should be the same as the highest validation

score printed in the previous step. Print evaluation scores (intent accuracy and slot micro F1) on the test set.

- **Train and select the AI-enhanced checkpoint**: After collaboration with AI, train your updated system using training data only. Retain epoch-level training/validation losses, validation results for each epoch, elapsed time, and learning curves. Save the checkpoint with the highest validation score (intent accuracy, slot micro F1, or combined, depending on your choice). Print its corresponding epoch index, validation score, and checkpoint filename.
- **Perform final evaluation on AI-enhanced checkpoint**: Load the checkpoint saved from the previous step. Without further training, evaluate the checkpoint on the validation and test splits. Your validation score should be the same as the highest validation score printed in the previous step. Print evaluation scores (intent accuracy and slot micro F1) on the test set.

What to include in the 2-page report

- **Baseline and improved systems**: describe the model pipelines before and after collaboration with AI. Clearly identify what changed and why.
- **Experimental results**: Report validation and test performance for the baseline, and final improved system in a comparison table. The table should follow the following format:

Model version	Split	Total # parameters	Total # trainable parameters	Intent accuracy (%)	Slot micro-F1 (%)	Number of epochs
Stage-1 system	Validation	…	…	…	…	…
Stage-1 system	Test	Same as above	Same as above	…	…	Same as above
Stage-2 system	Validation	…	…	…	…	…
Stage-2 system	Test	Same as above	Same as above	…	…	Same as above

- **AI collaboration**: Identify the AI tool/model used. Summarize the weaknesses it helped identify, and the suggestions you accepted, modified, or rejected. Explain your group's reasoning and contributions beyond the AI output.

- **Analysis and lessons learned**: Explain what you learned from AI, how you verified its advice, and any unsuccessful experiments.
- **Error analysis and limitations**: Analyze at least two error cases for the AI-improved system. Explain likely causes. Discuss limitations of the final system and experiments, supporting your observations with the error cases, and suggest possible future improvements.

Submit one archive named Group_<ID>.zip. The zip file should only contain the three files specified in Section 5. Do not include the dataset archive, environments, cache files, or API credentials.

6. Assessment

Performance and efficiency is part of assessment (50%): the baseline's test performance and the improved system's test performance. A large gain from a deliberately weak baseline does not compensate for poor absolute performance. Assessment also covers critical thinking (10%), code clarity and reproducibility (10%), and report quality (30%).